Technical Analysis using R: Development Phase

Course: BDA400 – Data Science Tools and Techniques

Assignment: Assignment 5 – Technical Analysis using R (Development Phase)

Submission Document & Execution Guide

1. Project Overview & Execution Instructions

This project provides custom base R implementations for nine core financial technical analysis indicators. Each indicator has been written strictly using base R core functions (no external libraries like TTR or quantmod were used) in accordance with the project templates and pseudocode specifications.

How to Run the Code in RStudio

1. **File Setup**: Ensure all 9 .R files are located in your active R working directory:
    - sma.R
    - ema.R
    - macd.R
    - stdev.R
    - linreg.R
    - rsi.R
    - stoch_rsi.R
    - crossover.R
    - crossunder.R
2. **Master Test Execution**: You can source all files and run a full demonstration using the following script in RStudio:

R
```
# Master Test Script for BDA400 Assignment 5

# Source all indicator scripts
source("sma.R")
source("ema.R")
source("macd.R")
source("stdev.R")
source("linreg.R")
source("rsi.R")
source("stoch_rsi.R")
source("crossover.R")
source("crossunder.R")

# Sample Datasets
price_data <- c(10, 12, 15, 20, 18, 22, 25, 24, 21)
rsi_price_data <- c(45, 50, 48, 55, 52, 49, 58, 60, 65, 62)
```

```
arr1 <- c(10, 12, 15, 20, 18, 22, 25, 24, 21)
arr2 <- c(18, 20, 22, 18, 15, 12, 10, 11, 13)

# 1. Test SMA
cat("--- SMA (period = 3) ---\n")
print(sma(price_data, period = 3))

# 2. Test EMA
cat("\n--- EMA (period = 3) ---\n")
print(ema(price_data, period = 3))

# 3. Test MACD
cat("\n--- MACD (short=3, long=6, signal=3) ---\n")
print(macd(price_data, short_period = 3, long_period = 6, signal_period = 3))

# 4. Test Standard Deviation
cat("\n--- Standard Deviation ---\n")
print(stdev(price_data))

# 5. Test Linear Regression
cat("\n--- Linear Regression (length=5, offset=0) ---\n")
print(linreg(price_data, regressionLength = 5, regressionOffset = 0))

# 6. Test RSI
cat("\n--- RSI (period = 5) ---\n")
print(rsi(rsi_price_data, period = 5))

# 7. Test Stochastic RSI
cat("\n--- Stochastic RSI (period=5, k=3, d=3) ---\n")
print(stoch_rsi(rsi_price_data, period = 5, k_period = 3, d_period = 3))

# 8. Test Crossover
cat("\n--- Crossover ---\n")
print(crossover(arr1, arr2))

# 9. Test Crossunder
cat("\n--- Crossunder ---\n")
print(crossunder(arr1, arr2))
```

2. Technical Indicator Descriptions & Implementation Details

1. Simple Moving Average (sma.R)

- **Logic**: Computes the arithmetic mean of data points over a rolling sliding window of size period.
- **Implementation Summary**: Iterates through indices from 1 to length(data) - period + 1, slices the target window using vector indexing, and calculates the mean using sum() / period. Inputs smaller than period trigger a validation error.

2. Exponential Moving Average (ema.R)

- **Logic**: Applies an exponential weighting scheme to prioritize recent price movements.
- **Implementation Summary**: Calculates a multiplier $\frac{2}{\text{period} + 1}$. The first data point serves as the initial EMA seed value ($\text{EMA}_1 = \text{data}[1]$). Subsequent values are computed recursively using $\text{EMA}_i = (\text{data}[i] - \text{EMA}_{i-1}) \times \text{multiplier} + \text{EMA}_{i-1}$.

3. Moving Average Convergence Divergence (`macd.R`)

- **Logic**: Evaluates trend direction and momentum by calculating the divergence between short-term and long-term EMAs.
- **Implementation Summary**: Calls `ema()` twice to derive the short-term and long-term EMAs. Subtracts long EMA from short EMA to form the MACD line, applies `ema()` on the MACD line to generate the signal line, and calculates the histogram as $\text{MACD Line} - \text{Signal Line}$. Returns all three series inside a structured R `list`.

4. Standard Deviation (`stdev.R`)

- **Logic**: Quantifies the dispersion or variability of a series around its mean.
- **Implementation Summary**: Computes the mean $\mu = \frac{\sum x}{N}$, calculates element-wise deviations $(x_i - \mu)$, squares the deviations, computes the variance $\sigma^2 = \frac{\sum (x_i - \mu)^2}{N}$, and takes the square root (`sqrt()`) to return the population standard deviation.

5. Linear Regression (`linreg.R`)

- **Logic**: Fits an ordinary least squares regression line ($y = m \cdot x + b$) across a specified subset window of price data.
- **Implementation Summary**: Validates boundary limits (`regressionLength` and `regressionOffset`), extracts the window subset, calculates mean index ($\bar{x}$) and mean price ($\bar{y}$), computes slope $m = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sum (x - \bar{x})^2}$ and intercept $b = \bar{y} - m\bar{x}$, and returns predicted values alongside slope and intercept in a list.

6. Relative Strength Index (`rsi.R`)

- **Logic**: Measures the speed and magnitude of recent price changes to evaluate overbought or oversold conditions.
- **Implementation Summary**: Uses `diff()` to calculate element differences, separates positive gains from absolute losses, computes initial average gain/loss over the first period, and updates subsequent periods using Wilder's smoothing method. Produces normalized values on a scale from 0 to 100.

7. Stochastic RSI (`stoch_rsi.R`)

- **Logic**: Applies the Stochastic Oscillator formula to RSI values to increase sensitivity to price momentum.
- **Implementation Summary**: Calls rsi() to generate the base RSI series, extracts valid non-NA bounds (RSI_{min} and RSI_{max}), calculates raw $\%K$ values using $\frac{\text{RSI} - \text{RSI}_{\text{min}}}{\text{RSI}_{\text{max}} - \text{RSI}_{\text{min}}}$, and applies sma() to derive the $\%K$ line and $\%D$ signal line.

8. Crossover Function (crossover.R)

- **Logic**: Identifies moments when a primary series crosses above or below a benchmark series.
- **Implementation Summary**: Compares element transitions between consecutive indices ($i-1$ and i). Returns "Up" when arr1 transitions from below/equal to strictly above arr2, "Down" when arr1 transitions from above/equal to strictly below arr2, and "None" otherwise.

9. Crossunder Function (crossunder.R)

- **Logic**: Detects specific events where a primary series drops strictly below a target series.
- **Implementation Summary**: Checks sequential array positions. Assigns "True" when arr1[i] < arr2[i] and arr1[i-1] >= arr2[i-1], returning "False" for all other points.